Experiment No.: 02

Aim : Implement A* Search for 8 puzzle problem **.**

Operating System recommended :- 64-bit Windows OS

and Linux

Theory:-

A* Search

- A* search is the most commonly known form of best-first search. It uses heuristic function h(n), andcost to reach the node n from the start state g(n). It has combined features of UCS and greedy best first search, by which it solve the problem efficiently.

- A* search algorithm finds the shortest path through the search space using the heuristic function.This search algorithm expands less search tree and provides optimal result faster.

- Algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).

The Basic Concept of A* Algorithm

- A heuristic algorithm sacrifices optimality, with precision and accuracy for speed, to solve problemsfaster and more efficiently.

- All graphs have different nodes or points which the algorithm has to take, to reach the final node. Thepaths between these nodes all have a numerical value, which is considered as the weight of the path. The total of all paths transverse gives you the cost of that route.

- Initially, the Algorithm calculates the cost to all its immediate neighboring nodes,n, and chooses the one incurring the least cost. This process repeats until no new nodes can be chosen and all paths havebeen traversed. Then, you should consider the best path among them. If f(n) represents the final cost,then it can be denoted as :

  f(n) = g(n) +

  h(n),where :

  g(n) = cost of traversing from one node to another. This will vary from node to node

  h(n) = heuristic approximation of the node's value. This is not a real value but an approximation cost

How Does the A* Algorithm Work?

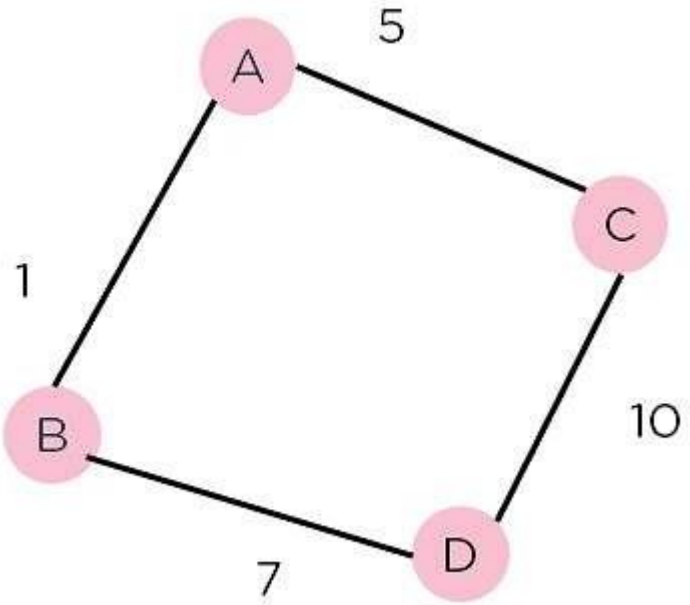


Figure of Weighted Graph

Consider the weighted graph depicted above, which contains nodes and the distance between them. Let's sayyou start from A and have to go to D.

Now, since the start is at the source A, which will have some initial heuristic value. Hence, the results

aref(A) = g(A) + h(A)

f(A) = 0 + 6 = 6

Next, take the path to other neighbouring vertices :

f(A-B) = 1 + 4

f(A-C) = 5 + 2

Now take the path to the destination from these nodes, and calculate the

weights :f(A-B-D) = (1+ 7) + 0

f(A-C-D) = (5 + 10) + 0

It is clear that node B gives you the best path, so that is the node you need to take to reach the destination.

Algorithm of A* search:

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if noden is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' andplace into Open list.

Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer whichreflects the lowest g(n') value.

Step 6: Return to Step 2.

Advantages:

1. A* search algorithm is the best algorithm than other search algorithms.

2. A* search algorithm is optimal and complete.

3. This algorithm can solve very complex problems.

Disadvantages:

1. It does not always produce the shortest path as it mostly based on heuristics and approximation.

2. A* search algorithm has some complexity issues.

3. The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it isnot practical for various large-scale problems.

```
# A* Search for 8 Puzzle

start = [1, 2, 3,
4, 0, 6,
7, 5, 8]

goal = [1, 2, 3,
4, 5, 6,
7, 8, 0]

def heuristic(state):
count = 0
for i in range(9):
if state[i] != 0 and state[i] != goal[i]:
count += 1
return count

def get_neighbors(state):
neighbors = []
```

```
pos = state.index(0)

moves = {
0:[1,3], 1:[0,2,4], 2:[1,5],
3:[0,4,6], 4:[1,3,5,7], 5:[2,4,8],
6:[3,7], 7:[4,6,8], 8:[5,7]
}

for m in moves[pos]:
new_state = state[:]
new_state[pos], new_state[m] = new_state[m], new_state[pos]
neighbors.append(new_state)

return neighbors

open_list = [(heuristic(start), start)]
visited = []

while open_list:
open_list.sort()
cost, current = open_list.pop(0)

print(current)

if current == goal:
print("\nGoal Reached")
break

visited.append(current)

for neighbor in get_neighbors(current):
if neighbor not in visited:
h = heuristic(neighbor)
open_list.append((h, neighbor))
```

Sample Output

```
[1, 2, 3, 4, 0, 6, 7, 5, 8]
[1, 2, 3, 4, 5, 6, 7, 0, 8]
[1, 2, 3, 4, 5, 6, 7, 8, 0
```

Conclusion:

In this Experiment, an introduction to the powerful search algorithm', we learned about everything about the algorithm and saw the basic concept behind it. Also implement the algorithm in python/Java.